



Advanced Programming

Java Collections

Framework

Collections

- A *collection* is an object that groups multiple elements into a single unit.
- *Vectors, Lists, Stacks, Sets, Dictionaries, Trees, Tables, etc.*
- Promotes software reuse
- Reduces programming effort
- Increases program speed and quality
- Benefits from *polymorphic algorithms*
- Uses *generics*

Collections Framework

- Interface



- Abstract class



- Concrete implementation

List

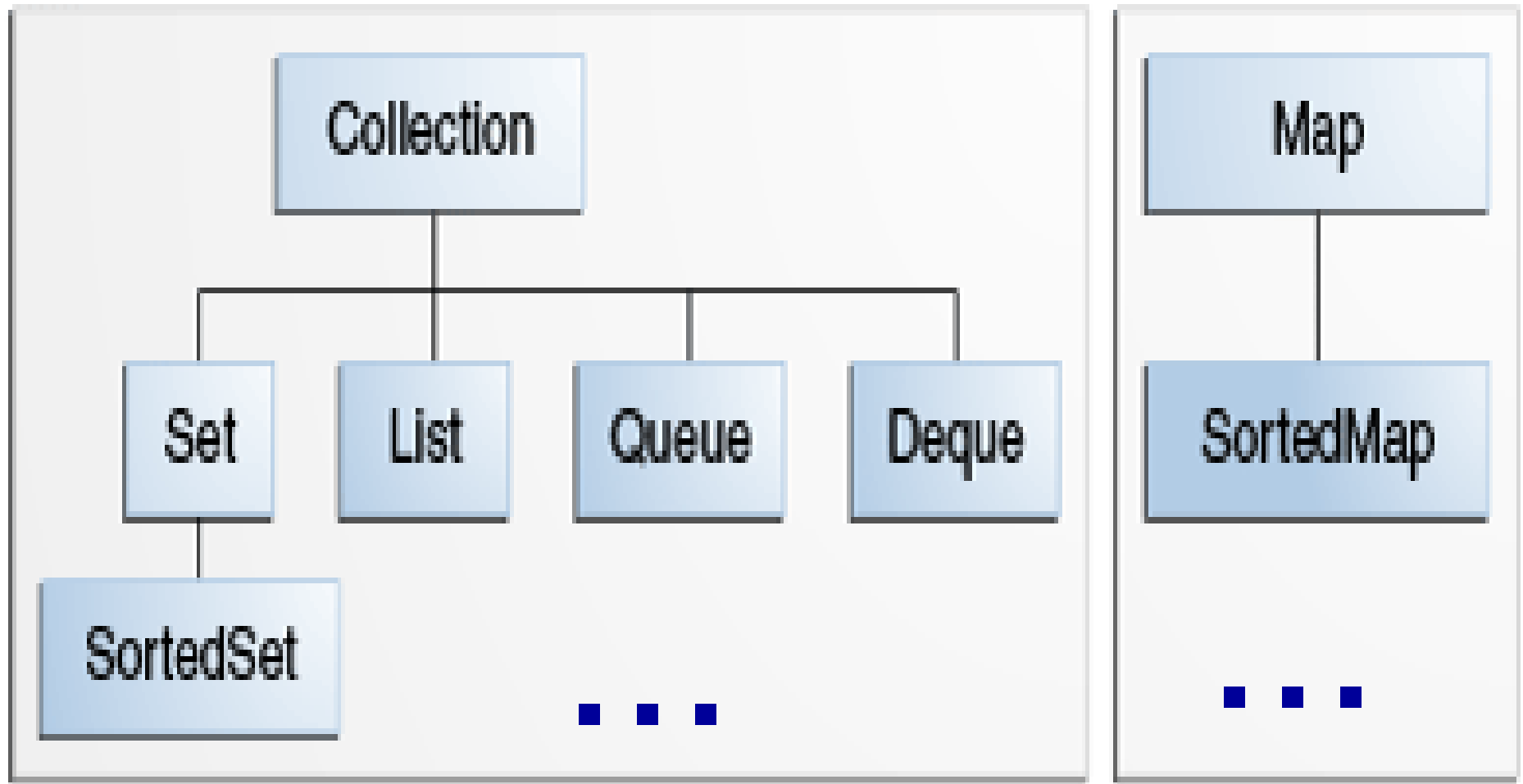


AbstractList



ArrayList
LinkedList
Vector

The Core Collection Interfaces



Implementations

Interfața	Hash	Array	Tree	Linked	Hash+Linked
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList Vector		LinkedList	
Queue		ArrayBlocking Queue		LinkedList, ConcurrentLin kedQueue	
Deque		ArrayDeque		LinkedDeque	
Map	HashMap Hashtable		TreeMap		LinkedHashMap



The *Collection* interface

- A collection represents a group of objects (elements)
- Some collections are *indexed*, others are not.
- Some collections allow *duplicate* elements, others do not.
- Some are *ordered* and others unordered.
- Some are *thread-safe*, others are not.
- All collections are *iterable*:
 - `public interface Collection<E> extends Iterable<E>`
- Dedicated subinterfaces: `List`, `Set`, `Queue`, etc.
- Many (indirect) implementing classes.

Common operations

```
public interface Collection<E> extends Iterable<E> {

    int size();
    boolean isEmpty();
    boolean contains(Object o);
    Iterator<E> iterator();

    Object[] toArray();
    <T> T[] toArray(T[] a);
    default <T> T[] toArray(IntFunction<T[]> generator) {
        return toArray(generator.apply(0));
    }

    boolean add(E e);
    boolean remove(Object o);
    boolean containsAll(Collection<?> c);
    boolean addAll(Collection<? extends E> c);
    boolean removeAll(Collection<?> c);
    boolean retainAll(Collection<?> c);
    void clear();
    ...
}
```

Lists: ArrayList, LinkedList

- A *list* is an indexed collection (sequence).
- Each element has a position (integer), starting from 0. Just like an array.
- Duplicates **are** allowed.
- A list can be implemented in various ways

```
List<String> list = new ArrayList<>();
```

```
List<Number> list = new LinkedList<>();
```

Bad practice:

Using implementations as reference types:

```
ArrayList<String> list = new ArrayList<>();
```

Using raw generics:

```
List list = new ArrayList();
```


Using a *List*

```
List<String> list = new ArrayList<>();  
  
list.add("a");  
  
list.add("b");  
  
System.out.println(list); // [a, b];
```

```
assert list.size() == 2;  
  
assert list.get(0).equals("a");  
  
assert list.get(1).equals("b");  
  
assert list.indexOf("a") == 0;
```

```
list.remove("b");
```

```
assert !list.contains("b");
```

```
assert list.contains("a");
```

indexOf, contains use the *equals* method of the elements!

Iterating Over a List

- As an indexed collections

```
for (int i=0, n=list.size(); i < n; i++ ) {  
    System.out.println( list.get(i) );  
}
```

- *Iterator and Enumeration*

```
for (Iterator it = list.iterator(); it.hasNext(); ) {  
    System.out.println(it.next());  
    it.remove();  
}
```

- *for-each*

```
List<Student> students = new ArrayList<>();  
...  
for (Student student : students) {  
    student.setGrade(10);  
}
```

Under the Hood

- **ArrayList**<E> implements **List**<E>

```
private static final int DEFAULT_CAPACITY = 10;  
Object[] elementData;  
private int size;
```

- **LinkedList**<E> implements **List**<E>

```
int size = 0;  
Node<E> first;  
Node<E> last;
```

```
private static class Node<E> {  
    E item;  
    Node<E> next;  
    Node<E> prev;  
    ...  
}
```

ArrayList or LinkedList?

```
public class TestList {
    private final static int N = 100_000;
    public void testAdd(List<Integer> list) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            list.add(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Add: " + (t2 - t1));
    }
    public void testGet(List<Integer> list) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            list.get(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Get: " + (t2 - t1));
    }
    public void testRemove(List<Integer> list) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            list.remove(0);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Remove : " + (t2 - t1));
    }
    public void runTests(List<Integer> list) {
        testAdd(list); testGet(list); testRemove(list);
    }
    public static void main(String args[]) {
        TestList app = new TestList();
        app.runTests(new ArrayList<>());
        app.runTests(new LinkedList<>());
    }
}
```

	ArrayList	LinkedList
add	6 ms	8 ms
get	3 ms	<u>4320</u> ms
remove	<u>868</u> ms	6 ms

Conclusion: Choosing a certain implementation depends on the nature of the problem being solved.

The *Set* interface

- A collection that contains no duplicate elements (models the mathematical set abstraction).

More formally, sets contain no pair of elements $e1$ and $e2$ such that $e1.equals(e2)$, and at most one null element.

- Sets are not indexed! We use *iterators* or *for-each* in order to traverse the set.
- A set can be implemented in various ways

```
Set<String> list = new HashSet<>();
```

```
Set<Number> list = new TreeSet<>();
```

Under the Hood

- **HashSet**<E> implements Set<E>

```
HashMap<E, Object> map;
```

Uses the *hashCode* of an Object.

This class offers **constant time** performance for the basic operations (add, remove, contains and size), assuming the hash function disperses the elements properly among the buckets.

- **TreeSet**<E> implements Set<E>

```
private NavigableMap<E, Object> m; //a specialized SortedMap
```

The elements are sorted using their natural ordering, or by a *Comparator* provided at set creation time

This implementation provides guaranteed **log(n) time** cost for the basic operations (add, remove and contains).

Using a Set

```
Set<String> set = new HashSet<>();  
set.add("b"); set.add("a"); set.add("a");  
System.out.println(set); // [a, b]
```

No guarantees about order.

```
assert set.size() == 2;  
assert set.get(0).equals("a");
```

```
set.remove("b");  
assert set.contains("a");
```

indexOf, contains use the *equals* method of the elements.

```
for(String s : set) {  
    System.out.println(s);  
}
```

ArrayList or HashSet?

```
public class TestSet {
    final static int N = 100_000;
    public void testAdd(Collection<Integer> collection) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            collection.add(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Add: " + (t2 - t1));
    }
    public void testIterate(Collection<?> collection) {
        long t1 = System.currentTimeMillis();
        for (Object obj : collection) {
            obj.toString(); //do something
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Iterate: " + (t2 - t1));
    }
    public void testContains(Collection<?> collection) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            collection.contains(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Contains: " + (t2 - t1));
    }
    public void runTests(Collection<Integer> collection) {
        testAdd(collection); testIterate(collection); testContains(collection);
    }
    public static void main(String args[]) {
        TestSet app = new TestSet();
        app.runTests(new ArrayList<>());
        app.runTests(new HashSet<>());
    }
}
```

	ArrayList	HashSet
add	6 ms	26 ms
iterate	51 ms	12 ms
contains	<u>3997</u> ms	7 ms
<i>Memory</i>	low	high

Conclusion: Choosing a certain implementation depends on the nature of the problem being solved.

The *Queue* interface

- A queue is a first-in-first-out (**FIFO**) linear collection; It has a *head* and a *tail*.
- Provides additional *insertion*, *extraction*, and *inspection* operations.

`add, remove, element` (throw exceptions)

`offer, poll, peek` (return special values)

- Example

```
Queue<String> q = new LinkedList<>();  
q.offer("a"); // same as q.add  
q.offer("b"); // adds elements at the tail  
assert q.peak().equals("a"); //returns the head  
assert q.poll().equals("a"); //returns and removes  
assert q.peak().equals("b");
```

The *PriorityQueue* class

- A queue based on a priority **heap**. The elements of the priority queue are ordered according to their natural ordering, or by a *Comparator*. The *head* of the queue is *the least element* with respect to the ordering.
- The implementation provides **$O(\log(n))$** time for the enqueueing and dequeueing methods, **linear time** for the *remove* and *contains* methods; and **constant time** for the retrieval methods (*peek*, *element*, *size*).
- Example:

```
PriorityQueue<Integer> pQueue = new PriorityQueue<>();  
pQueue.add(3); pQueue.add(1); pQueue.add(2);  
assert pQueue.peek() .equals(1);
```

Stacks and Deques

- The *Stack* class represents a last-in-first-out (**LIFO**) stack of objects. Obsolete, just like *Vector*.
- **Deque** is a linear collection that supports element insertion and removal **at both ends** ("double ended queue").
- Deques can be used to implement LIFO collections:

```
Deque<Integer> stack = new ArrayDeque<>();  
stack.push(1); //same as addFirst  
stack.push(2);  
assert stack.peekFirst().equals(2); //same as peek  
assert stack.peekLast().equals(1);
```

The *Map* interface

- An object that maps keys to values.
- It is similar to describing a *dictionary*.
- `public interface Map<K, V> { ... }`
- A map cannot contain duplicate keys; each key can map to at most one value.
- **Example:** `Map<Country, City>`

<u>Key</u>	<u>Value</u>
Romania	→ Bucharest
France	→ Paris

- Implementations:

```
Map<Country, City> capitals = new HashMap<>();
```

```
Map<Word, List<Definition>> dict = new TreeMap<>();
```

Using a *Map*

```
Map<String, String> map = new HashMap<>();  
map.put("Romania", "Bucharest");  
map.put("France", "Paris");  
System.out.println(map);  
//Romania=Bucharest, France=Paris}  
  
assert map.get("France").equals("Paris");  
assert map.containsKey("Romania");  
  
for(String country : map.keySet()) {  
    System.out.println(map.get(country));  
}
```

Under the Hood

- *HashMap* can achieve an average time complexity of $O(1)$ for the *put* and *get* operations and space complexity of $O(n)$.
- Instead of iterating over all its elements, *HashMap* attempts to calculate the position of a value based on its key.
- *HashMap* stores elements in so-called buckets and the number of buckets is called *capacity*.

```
Node<K,V>[] table;
```

```
Set<Map.Entry<K,V>> entrySet;
```

- The *index* in the *table* array is computed using the *hashCode* method of the key object.
- Inside a bucket, values are stored either in a *list* (for less than 8 elements) or in a *balanced tree*. The data structure changes dynamically. The performance of iterating: $O(\log |\text{bucket}|)$.

hashCode and *equals*

```
public class Player {  
    private String name;  
    // ...  
  
    @Override  
    public int hashCode() {  
        int hash = 7;  
        hash = 67 * hash + Objects.hashCode(this.name);  
        return hash;  
    }  
  
    @Override  
    public boolean equals(Object obj) {  
        if (this == obj) return true;  
        if (obj == null) return false;  
        if (!(obj instanceof Player)) return false;  
        Player other = (Player) obj;  
        return Objects.equals(this.name, other.name);  
    }  
  
    // ...  
}
```

When a *hashCode()* comparison returns false, the *equals()* method must also return false.

The importance of *equals*

Assume there is another property *Integer id*;

```
public boolean equals(Object obj) {  
    if (this == obj) return true;  
    if (obj == null) return false;  
    if (!(obj instanceof Player)) return false;  
    Player pers = (Player) other;  
    return Objects.equals(this.id, other.id);  
}
```

```
Map<Player, String> map = new HashMap<>();  
map.put(new Player("Messi"), "Barcelona");  
map.put(new Player("Ronaldo"), "Juventus");
```

```
System.out.println(map);
```

```
{Messi=Juventus}
```


Polymorphic Algorithms

`java.util.Collections`

- `sort`
- `shuffle`
- `binarySearch`
- `reverse`
- `fill`
- `copy`
- `min`
- `max`
- `swap`
- `enumeration`
- `unmodifiableCollectionType`

```
List<String> immutablelist = Collections.unmodifiableList(list);  
immutablelist.add("Oops...?!");
```
- `synchronizedCollectionType`



What DesignPattern?

var

Type inference for local variables

- The explicit type can be replaced by the reserved type name **var** for local variable declarations that have initializers.

String str = "Hello";



var str = "Hello World";

- Explicit vs. Implicit, Verbosity vs. Readability
- Examples

```
var stringList = List.of("a", "b", "c");
```

```
var personList = new ArrayList<Person>();  
for(var person : personList) { }
```

```
var companyToEmployees = new HashMap<String, List<String>>();  
for (var entry: companyToEmployees.entrySet()) {  
    var employees = entry.getValue();  
}
```

```
var itemQueue = new PriorityQueue<>(); // PriorityQueue<Object>  
var x = null;
```



Advanced Programming

Stream API

The Context

- We have **data** stored in various collections.
- We must perform various operations on that data: *sorting, filtering, grouping, mapping, etc.*
- Using only iterators, we may repeat the same constructions over and over again.
- We need a mechanism that is: *terser (concise, declarative), functional, less mutable, loose-coupled, more performant(?)* when working with collections.

Example

```
int sum = 0;
for(Item e : items) {
    if (e.getWeight() <= 10) {
        continue;
    }
    if (!e.isAvailable()) {
        continue;
    }
    sum += e.getPrice();
}
```

What is a *stream*?

- A *stream* is a sequence of elements supporting sequential and parallel aggregate operations.
- Elements can be *objects* or *primitive* values (int, long, double). See *java.util.stream* package.
- Example

```
int sum = items.stream()  
    .filter(e -> e.getWeight() > 10)  
    .filter(e -> e.isAvailable())  
    .mapToInt(e -> e.getPrice())  
    .sum();
```

Stream Pipelines

- To perform a computation, stream operations are composed into a **stream pipeline (query)**.
- A stream pipeline consists of a **source** (which might be an array, a collection, a generator function, an I/O channel, etc), zero or more **intermediate operations** (which transform a stream into another stream, such as *filter(Predicate)*), and a **terminal operation** (which produces a result or side-effect, such as *count()* or *forEach(Consumer)*).
- Streams are **lazy**; computation on the source data is only performed when the terminal operation is initiated, and source elements are consumed only as needed.
- **Collections** are concerned with the efficient management of and their elements, **streams** are concerned with declaratively describing their source and the computational operations which will be performed.

Functional Interfaces

- A functional interface is any interface that **contains only one abstract method**.
- Instead of using an anonymous class, you use a *lambda expression*, omitting the name of the interface and the name of the method.

```
Arrays.sort(p, (Person p1, Person p2) -> {  
    return p1.getName().compareTo(p2.getName());  
});
```

```
@FunctionalInterface  
public interface Comparator { ...}
```


java.util.function packages

Functional interfaces, having various methods:

- **Function**
 - *apply*: accepts one argument and produces a result
- **Predicate**
 - *test*: boolean-valued function of one argument
- **Consumer**
 - *accept*: a single input argument and returns no result
- **Supplier**
 - *get*: does not take any argument and produces a value
- ...

Instances of these types will be usually created using **lambda-expressions** and **method references**.

Under the Hood

```
int sum = Arrays.asList(1, 2, 3, 4, 5).stream()
    .filter(x -> x >= 3)
    .mapToInt(Integer::intValue)
    .sum();
```

- *filter*

```
Stream<T> filter(Predicate<? super T> predicate);
```

- *mapToInt*

```
IntStream mapToInt(ToIntFunction<? super T> mapper);
```

- *sum*

```
int sum();
```

Explicit Use of Functional Interfaces

- *Predicate*

```
Predicate<Integer> pred1 = x -> (x > 2);
```

```
Predicate<Integer> pred2 = x -> (x < 5);
```

```
System.out.println(pred1.and(pred2).negate().test(1));
```

- *Function*

```
Function<String, Integer> fun1 = str -> str.length();
```

```
Function<Integer, Integer> fun2 = x -> x*x;
```

```
System.out.println(fun1.andThen(fun2).apply("Hello"));
```

- ...

Stream Creation

- *Stream.empty*
- *Collection.stream*

```
Stream<String> s = collection.stream();
```

- *Stream.of*

```
Stream<String> s = Stream.of("a", "b", "c");
```

- *Arrays.stream*

```
int[] arr = {1,2,3};
```

```
IntStream s = Arrays.stream(arr);
```

- *range, rangeClosed*

```
LongStream longStream = LongStream.range(0, 10);
```

- ...

Intermediate Operations

filter, map, reduce, distinct, sorted, skip, etc.

- Intermediate operations are **applied on a source** stream.
- Intermediate operations return **a new modified stream**.

```
Stream<String> s = Stream.of("a", "b").skip(1);
```

- Operations can be **chained**

```
stream.sorted().distinct().skip(1);
```

- Intermediate operations are ended by a **single terminal operation**:

```
long x = Stream.of("a", "a").distinct().count();
```

Terminal Operations

sum, count, min, max, average, forEach, collect, etc.

- Are **applied on a (intermediate)** stream.
- Terminal operations return **a value** (not a stream).

```
Stream.of("a", "b").count();
```

```
Stream.of("Hello", "World").forEach(System.out::println);
```

```
persons.stream()  
    .filter(p -> p.getAge() >= 18)  
    .mapToInt(Person::getAge)  
    .average()  
    .getAsDouble();
```

- Terminal operations cannot be **chained**

Iterating, Filtering and Matching

stream → filter → findFirst, findAny, allMatch, anyMatch, noneMatch, etc.

- Substitute for: **for-each** and **while** loops.
- “Classical” iteration

```
for (Person p : persons) {  
    if (p.getAge() >= 18 && p.getName().endsWith("escu")) {  
        return p;  
    }  
}  
return null;
```

- “Fancy” stream-based iteration

```
persons.stream()  
    .filter(p -> p.getAge() >= 18)  
    .filter(p -> p.getName().endsWith("escu"))  
    .findFirst()  Optional<Person>  
    .orElse(null);
```

Optional

Tired of Null Pointer Exceptions?

- A *container (wrapper)* object which may or may not contain a non-null value. If a value is present, *isPresent()* will return true and *get()* will return the value.
- What problem is it trying to solve?

```
String countryName =  
person.getAddress().getCountry().getName();
```

- Important for stream intermediate operations.
- Groovy has a *safe navigation operator*:

```
person?.getAddress()?.getCountry()?.getName();
```

- Kotlin has a type system that distinguishes between *references that can hold null and those that can not*.

```
var a: String = "abc"           var b: String? = "abc"  
a = null //compilation error    b = null // ok
```


Using Optional

- Explicitly

```
String name = "John"; //may also be null
Optional<String> opt = Optional.ofNullable(name);
```

```
assert opt.isPresent(); assert !opt.isEmpty();
assert opt.get().equals(name);
```

```
opt.ifPresent(System.out::println);
int len = opt.orElse("").length();
```

- Cascading operations

```
String countryName = person.map(Person::getAddress)
    .map(Address::getCountry)
    .map(Country::getName)
    .orElse("UNKNOWN");
```

Reducing

- Reducing a sequence of elements to some value according to a specified function.
- The method **reduce** takes two parameters: a **start value**, and an **accumulator function**.

```
List<Integer> numbers = Arrays.asList(1, 2, 3);
```

```
Integer reduced = numbers  
    .stream()  
    .reduce(0, (a, b) -> a + b);
```

```
//same as  
Integer reduced = numbers.stream()  
    .reduce(0, Integer::sum);
```

Mapping

- Converting elements of a *Stream*, by applying a special function to them, and collecting the new elements into another *Stream*.

```
items.stream()  
    .map(item -> item.getProduct())  
    .mapToDouble(product -> product.getPrice())  
    .average().getAsDouble();
```

- The *flatMap* returns a stream consisting of the results of replacing each element of this stream with the contents of a mapped stream produced by applying the provided mapping function to each element.

```
orders.flatMap(order -> order.getItems().stream())  
    .forEach(System.out::println);
```

Collecting

- Converting a stream to a *Collection* or a *Map* .
- The utility class *Collectors* provides a solution for almost all typical collecting operations.
- Collecting to a *List*, *Set*, etc.

```
List<String> names = persons.stream()  
    .map(p -> p.getName())  
    .collect(Collectors.toList());
```

- Collecting to a *String*

```
String listToString persons.stream()  
    .map(Person::getName)  
    .collect(Collectors.joining(", ", "[", "]"));
```

Sorting

- Using the natural order

```
List<String> sortedList = list.stream()  
    .sorted().collect(Collectors.toList());
```

- Using a *Comparator*

```
List<Person> sortedList = persons.stream()  
    .sorted(Comparator.comparingInt(User::getAge))  
    .collect(Collectors.toList());
```

- Reversed

```
List<String> reversedList = list.stream()  
    .sorted().reversed().collect(Collectors.toList());
```

- ...

Grouping

```
Team barca = new Team("FC Barcelona");
Team real = new Team("Real Madrid");
Player[] players = {
    new Player("Lionel Messi", barca),
    new Player("Louis Suarez", barca),
    new Player("Antoine Griezman", barca),
    new Player("Karim Benzema", real),
    new Player("Eden Hazard", real)
};
```

a classifier *function* mapping
input elements to keys



```
Map<Team, List<Player>> teamPlayers =
    Arrays.stream(players)
        .collect(Collectors.groupingBy(Player::getTeam));
```

```
System.out.println(teamPlayers);
```

```
// {Real Madrid=[Karim Benzema, Eden Hazard],
// FC Barcelona=[Lionel Messi, Louis Suarez, Antoine Griezman]}
```

Counting and Summing

- Counting

```
Map<String, Long> counting = items.stream()  
    .collect(Collectors.groupingBy(  
        Item::getName,   
        Collectors.counting()));
```

a classifier function mapping input elements to keys

a collector implementing the downstream reduction

- Summing

```
Map<String, Integer> sum = items.stream()  
    .collect(Collectors.groupingBy(  
        Item::getName,  
        Collectors.summingInt(Item::getQty)));
```

- ...

Parallel Streams

- Stream pipelines may execute either **sequentially (default)** or in **parallel**.
- `stream.parallel()` creates a parallel stream.
- Advantage: it uses a **multi-threaded approach** in order to execute the functions inside the pipeline, making better use of multi-core processors.
- Disadvantage: it has a much higher overhead compared to a sequential one. It pays off only if there is a large number of independent tasks.

Using a *parallelStream*

//Sequential

```
IntStream stream1 = IntStream.rangeClosed(1, 10);
```

```
stream1.forEach(System.out::println);
```

```
// 1 2 3 4 5 6 7 8 9 10
```

//Parallel

```
IntStream stream2 = IntStream.rangeClosed(1, 10);
```

```
stream2.parallel().forEach(System.out::println);
```

```
// 7 6 3 2 5 9 1 10 4 8
```

Improving Performance

```
private boolean isPrime(long number) {  
    long d = 2;  
    while (d * d <= number) {  
        if (number % (d++) == 0) return false;  
    }  
    return true;  
}
```

```
private void test() {  
    int n = 100_000_000; //Looking for a prime number in a large array  
    long arr[] = new long[n];  
    for (int i = 0; i < n - 1; i++) {  
        arr[i] = 100003 * 100003;  
    }  
    arr[n - 1] = 2;  

```

```
    long nr1 = Arrays.stream(arr)  
        .filter(this::isPrime).findAny().getAsLong();
```

7.435 s

```
    long nr2 = Arrays.stream(arr).parallel()  
        .filter(this::isPrime).findAny().getAsLong();
```

2.677 s

```
    }  
}
```

Recap

- **Stream** - sequence of elements supporting sequential and parallel aggregate operations.
- **Pipeline** - a sequence of aggregate operations.

```
persons.stream()  
    .filter(p -> p.getAge() >= 18)  
    .filter(p -> p.getName().endsWith("escu"))  
    .forEach(s -> System.out.println(s.getName()));
```

- **Mapping** and **Terminal** ops.

```
double averageAge = persons.stream()  
    .filter(p -> p.getAge() >= 18)  
    .mapToInt(Person::getAge)  
    .average()  
    .getAsDouble();
```

- **Source**
array, collection, ...
- **Intermediate** operations
filter, distinct, sorted,...
- **Mapping** operations
map, mapToInt, ...
- **Terminal** operations
average, min, max,...